# Revisões

Histórico das rodadas de revisão da proposta; todas as decisões abaixo já estão incorporadas na aba principal.

| Rodada | Origem | Principais mudanças |
|---|---|---|
| 3 | Ideias do autor | Simplicidade como requisito (segurança por padrão, cerimônia por exceção; camadas de adoção; métricas de simplicidade); tipo `money` com formato por idioma; projeto de exemplo em Rails e Vetra |
| 2 | Segunda avaliação do ChatGPT | Contexto assinado no RLS; `security_invoker` em views; FKs compostas; contexto na transação externa; cinco testes Sequel + Active Record; Effect Boundary; modo sombra movido para a fase 4; níveis de garantia; ADR-016 como central |
| 1 | Primeira avaliação do ChatGPT | Workflows por replay determinístico; isolamento em três níveis com Ruby::Box opcional; IR versionada e regras do grafo; RLS detalhado; reatividade com invalidação e ressincronização; MCP com delegação; LGPD como ferramentas de conformidade; modo serviço; métricas pelo tempo de mudança; roadmap reordenado |

Rodada 4 (ideia do autor): o Exemplo 2 — Blog mostrou o Vetra no caso simples, sem multiempresa, e revelou uma lacuna: a camada de UI no servidor ainda não tem especificação. Ela foi registrada nas questões em aberto.

Rodada 5 (ideia do autor): o Exemplo 3 — Chat confirmou a vantagem do Vetra na revogação de acesso em tempo real, mas expôs duas lacunas: o custo de renderizar para cada assinante e a falta de sinais efêmeros (presença, "digitando"). As duas foram registradas nas questões em aberto. Nesta rodada também entraram a árvore completa de projeto e os diagramas de arquitetura (núcleo e extensões, ciclo de vida da operação, modelos de dados).

Rodada 6 (decisão do autor): as lacunas do Exemplo 3 viraram três ADRs. O ADR-020 troca a reconsulta por assinante pela entrega incremental, com teste de escopo em memória, renderização por grupo de permissão e revalidação periódica. O ADR-018 especifica a camada de UI com Turbo como cliente no navegador. O ADR-019 define os sinais efêmeros com a mesma autorização das consultas live. A maturidade foi tratada como risco a mitigar, não como melhoria de projeto.

Rodada 7 (sugestões do Gemini e do ChatGPT para as pendências): entraram a revisão de autorização por trigger, os modos strict e balanced, o agrupamento em dois níveis, a ordem de eventos por xid8, a ejeção em três níveis, os módulos de UI, a presença por leases, a conexão LISTEN dedicada, a matriz de carga, o checklist do gate G4 e o ADR-021. Foram rejeitados: o sinalizador "verificado" em variável de sessão (Gemini), que permitiria forjar o tenant; a afirmação de que o crypto-shredding vale para backups anteriores; limites numéricos sem fonte para o NOTIFY; Ractor no caminho do filtro; e falha de boot por template ejetado, que ERB dinâmico não permite garantir.

Erros desta proposta corrigidos na rodada 7:

- Revogação descrita como "imediata"; agora é garantida por lote no modo strict, com a janela residual documentada.
- O CRUD gerado do Vetra permitia gravar `status: "won"` e publicar posts sem o comando; corrigido pelos campos de transição (ADR-021). O exemplo em Rails não tinha esse problema.
- `idempotent_by` descrito como garantia contra e-mail duplicado; ele só deduplica o handler.
- Renderização por grupo combinada com `can?` por autor no Exemplo 3, sem regra para isso; agora há partição por evento.

## Erros reconhecidos

- Persistência de Fibers apresentada como viável; substituída por histórico de eventos e replay.
- Evento sem consumidor tratado como erro de boot; agora eventos públicos são permitidos.
- Canário que duplicaria efeitos reais; agora roda em modo sombra com capacidades simuladas.
- "Isolamento provado por teste" e "LGPD por construção"; testes não são prova, e o framework oferece ferramentas, não conformidade automática.
- Afirmação de que uma gem com conexão própria não escaparia do RLS; a negação por padrão não impede falsificação de contexto, daí o contexto assinado.
- Contagem de linhas da comparação de código que subestimava o Rails.

## Onde a proposta divergiu do ChatGPT

- **Workflows:** manteve Ruby linear com replay determinístico, em vez de uma DSL de etapas; o ChatGPT aceitou na segunda rodada.

- **Roadmap:** manteve um spike descartável de reatividade na fase 1, para medir cedo o maior risco.
- **Modo sombra:** capacidades simuladas em vez de restringir a operações somente leitura.
- **Posicionamento:** plataforma por dentro, framework por fora, em vez de se apresentar como plataforma de aplicação.